

Practice Set: Inheritance in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define **Inheritance** in C++ and distinguish between a **Base Class** and a **Derived Class**. Outline the three major reasons why inheritance is essential in object-oriented software design.

Question 2: Explain the **Memory Allocation Mechanics** in RAM when a derived class object is instantiated. How are base class sub-objects and derived class sub-objects laid out in memory? Do private members of a base class occupy memory in a derived object?

Question 3: Detail the basic C++ syntax for class inheritance. Explain what happens if a programmer omits the explicit visibility mode in a class declaration (e.g., class B : A).

Question 4: Explain **Single Inheritance** and **Multilevel Inheritance** with appropriate class structure diagrams or textual representations and C++ code declaration syntax.

Question 5: Explain **Multiple Inheritance**, **Hierarchical Inheritance**, and **Hybrid Inheritance**. How does hybrid inheritance often form a diamond structure?

Question 6: Differentiate between the two types of class consumers: **User 1 (The Client / Instantiator)** and **User 2 (The Class Extender / Derived Class Author)**. Explain how the protected access specifier serves User 2 while maintaining encapsulation against User 1.

Question 7: Fill out and explain the **Visibility Mode Transformation Matrix** for public, protected, and private inheritance modes across base member categories (private, protected, public).

Question 8: Explain what happens to the visibility of public and protected base class members when a derived class inherits using **private inheritance**. Can a subclass of this derived class access those inherited members?

Question 9: Write a complete C++ program demonstrating **Single Inheritance** with a protected access specifier. Show a base class A containing a private member and a protected setter method, and a derived class B calling that protected setter to update base state.

Question 10: Write a complete C++ program demonstrating **Multiple Inheritance** where a Car class inherits publicly from both an Engine class and a Radio class. Instantiate a Car object in main() and invoke methods inherited from both base classes.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Are private members of a base class inherited by a derived class?

Question 12 (Viva): What is the default visibility mode when deriving a class without specifying access (e.g., `class Derived : Base`)?

Question 13 (Viva): Can a client function like `main()` directly call a member function declared in the protected section of a base class?

Question 14 (Viva): Under which visibility mode do public base members remain public and protected base members remain protected in the derived class?

Question 15 (Viva): What is the contiguous RAM allocation layout order when an object of a derived class is created?

Question 16 (Viva): If Class B inherits from Class A using protected inheritance, what access level do public members of Class A acquire inside Class B?

Question 17 (Viva): What type of inheritance is represented when a single derived class inherits from two or more base classes simultaneously?

Question 18 (Viva): What type of inheritance is represented when multiple sibling derived classes inherit from a single common base class?

Question 19 (Viva): If Class B privately inherits Class A, can a further derived Class C (derived from B) access the inherited members of Class A?

Question 20 (Viva): What structural shape or hierarchy frequently arises when combining multilevel and multiple inheritance in C++?

Answers & Explanations

Answer: 01

Definition: Inheritance is a fundamental OOP pillar where a new class acquires the properties (data members) and behaviors (member functions) of an existing class.

Class Types:

Base Class (Parent/Super): The existing class whose members are inherited.

Derived Class (Child/Sub): The new class that inherits members from the base class.

Three Reasons for Inheritance:

Code Reusability: Derived classes directly inherit verified features without rewriting existing code.

Transitive Extensibility: System domain models can be extended without altering or recompiling base library source code.

Hierarchical Classification: Helps accurately model real-world taxonomy chains (e.g., Vehicle $\rightarrow$ Car $\rightarrow$ ElectricCar).

Answer: 02

RAM Layout: When a derived object is instantiated, the OS allocates a single contiguous RAM block. The base class sub-object data members are stored first at lower memory addresses, immediately followed by the derived class's own data members.

Private Member Allocation: Base class private members **do physically occupy space** inside the derived object's RAM layout. However, compiler access control rules render them hidden and inaccessible to derived class member functions.

Answer: 03

Syntax:

```
class BaseClass
{
    // Base class members
};
class DerivedClass : visibility_mode BaseClass
{
    // Derived class members
};
```

Default Mode Behavior: If visibility_mode is omitted, C++ defaults strictly to **private inheritance**.

Answer: 04

Single Inheritance: A derived class inherits directly from one base class.

Structure: Class A (Base) -----> Class B (Derived)

Syntax: class B : public A { ... };

Multilevel Inheritance: A class is derived from another class that is itself derived from a base class, forming a chain.

Structure: Class A (Grandparent) -----> Class B (Parent) ----> Class C (Child)

Syntax: class B : public A { ... }; then class C : public B { ... };

Answer: 05

Multiple Inheritance: A single derived class inherits simultaneously from two or more distinct base classes (e.g., class B : public A1, public A2).

Hierarchical Inheritance: Multiple child classes inherit from one common parent base class (e.g., class B1 : public A and class B2 : public A).

Hybrid Inheritance: A combination of two or more inheritance patterns (such as Multilevel and Multiple). It often forms a **Diamond Hierarchy** where two intermediate classes derive from a shared base, and a child class derives from both intermediate classes.

Answer: 06

User 1 (Client / Object Instantiator): Instantiates class objects in external functions like main(). User 1 can only access public members of the object.

User 2 (Class Extender / Derived Author): Extends base functionality by creating derived child classes.

Protected Role: protected members block direct access from User 1 (preventing external modification), but grant direct access to User 2 within derived class method bodies, balancing encapsulation and extensibility.

Answer: 07

Base Member Access	public Inheritance	protected Inheritance	private Inheritance
private	Not Inherited (Inaccessible)	Not Inherited (Inaccessible)	Not Inherited (Inaccessible)
protected	Remains protected in derived	Becomes protected in derived	Becomes private in derived
public	Remains public in derived	Becomes protected in derived	Becomes private in derived

Answer: 08

Visibility Shift: Under private inheritance, both public and protected base members convert to **private members** inside the derived class.

Subclass Impact: Subclasses of this derived class (User 2 of the derived class) **cannot access** those inherited members because they are now private inside the intermediate class.

Answer: 09

```
#include <iostream>
using namespace std;
class A
{
    private:
        int a; // Inaccessible to derived classes
    protected:
        // Accessible to derived Class B, hidden from main()
        void setValue(int k) {
            a = k;
        }
    public:
        void displayA() {
            cout << "Value of a (Base A): " << a << endl;
        }
};
// Class B inherits publicly from Class A
class B : public A {
    public:
        void setData(int x) {
            // Valid: Direct call to protected method of Base A
            setValue(x);
        }
};
int main()
{
    B obj;
    // obj.setValue(10); // ERROR: setValue is protected
    obj.setData(4); // Valid public method of B
    obj.displayA(); // Valid inherited public method of A
    return 0;
}
```

Answer: 10

```
#include <iostream>
using namespace std;
class Engine
{
    public:
```

```

    void startEngine()
    {
        cout << "Engine Started!" << endl;
    }
};
class Radio {
    public:
    void playMusic()
    {
        cout << "Playing Music!" << endl;
    }
};

class Car : public Engine, public Radio
{
    public:
    void drive()
    {
        cout << "Car is Driving..." << endl;
    }
};
int main()
{
    Car myCar;
    myCar.startEngine(); // Inherited from Engine
    myCar.playMusic();   // Inherited from Radio
    myCar.drive();       // Car's own method
    return 0;
}

```

Answer: 11

They occupy physical memory inside the object, but are compiler-restricted as **inaccessible** to derived class methods.

Answer: 12

private inheritance.

Answer: 13

No; protected members are inaccessible to external client functions like main().

Answer: 14

public inheritance.

Answer: 15

The **base class sub-object** is allocated first, followed immediately by the **derived class sub-object**.

Answer: 16

They become **protected members** inside Class B.

Answer: 17

Multiple Inheritance.

Answer: 18

Hierarchical Inheritance.

Answer: 19

No; because those members became private inside Class B, making them inaccessible to Class C.

Answer: 20

A **Diamond Hierarchy** (or Diamond structure).